Olozzo.com: Stripe Live Deployment Guide

Audience	Developer, server administrator / DevOps, project manager. Client tasks are in the Launch Checklist
Scope	Deploy the existing, sandbox-tested Stripe integration to production **without changing working payment code**, then activate live payments
Code inspected	Branch `stripe-integration-hardening`, uncommitted working tree on top of `3f49ae8`; remote `github.com/nishilwork11/olozzo` (inspected 29 Oct 2026)
Companion documents	Production Runbook (execution order) · Launch Checklist (sign-off) · Integration Guide (architecture) · Test/Live Mode Guide · Client Setup Guide

Nothing in this guide has been run against production. No server was contacted, no live key exists in this document, and no real payment was made.

Status labels: ✅ **Verified** (code read and exercised in sandbox QA on 29 Oct 2026) · 🟡 **Needs confirmation** · ❌ **Missing / blocker** · ❓ **Unknown** (information not in the repository).

Command classes: 🟢 **INSPECT** read-only · 🟡 **CONFIGURE** changes configuration or the repository, reversible · 🔴 **PRODUCTION** changes the live site or live database; needs approval and a maintenance window.

---

1. What was inspected

Item	Finding	Status
Framework	Next.js 15.5.28 (App Router: Route Handlers, Server Actions, Server Components), React 19.2.9, TypeScript. The frontend and backend are one Next.js application	✅
Runtime / package manager	Node.js (≥ 20 required by `stripe@23`; local 22.15.0); **npm** with `package-lock.json`	✅
Scripts	`build` = `prisma generate && next build` · `start` = `next start` · `typecheck` · `test` (Node test runner) · `db:push` · `setup` / `db:seed` / `db:reset` (wipe the database: **never on production**) · `db:seed-catalog` (additive catalog import, documented as safe on live; not part of this deployment)	✅
Database	PostgreSQL via Prisma 6.19.3 (`DATABASE_URL`); schema applied with `prisma db push`; no migration history	✅
Stripe SDK	`stripe` 23.0.1, pinned API version `2026-09-30.endive`	✅
Stripe architecture	**Hosted Stripe Checkout** (redirect). The server creates a Checkout Session per order; no Stripe.js or Elements; **no publishable key used**	✅
Deployment configuration	**None in the repository**: no Dockerfile, docker-compose, `vercel.json`, PM2/ecosystem, Procfile, systemd unit, Nginx/Apache config or CI workflow	✅ (absence verified)
Hosting provider	`.vercel` in `.gitignore` and the README ("Vercel, or any Node host"; "Neon via the Vercel Marketplace") **suggest** Vercel and Neon	❓ Unconfirmed
Production domain	Not documented. `olozzo.com` appears only in marketing copy and comments	❓
SSL / web server / process manager	Not in the repository	❓

Item	Finding	Status
Environment management	Local `.env` (git-ignored). Production method not documented	❓
Logging	`console.*` to stdout with prefixes `[stripe]`, `[checkout]`, `[payments]`, `[admin]`	✅
Monitoring / alerting / health check	None implemented	❌ (risk, not blocker)

2. Is the integration ready to deploy?

2.1 Payment behaviour (evidence: code review + sandbox QA, 29 Oct 2026)

Requirement	Evidence	Status
Test-mode payments work locally	Card, 3D Secure, decline + retry, cancel, SEPA success and failure, dispute; 45/45 webhook deliveries `200`	✅
Backend creates the expected Stripe objects	`createStripeCheckout()` in `src/lib/payments/stripe.ts`: `mode: payment`, EUR, one line item = server-calculated order total, `metadata.orderId`, idempotency key `checkout-session-<orderId>`, `expires_at` = 60-minute payment window. 30/30 sessions matched order totals	✅
Payment status verified securely	Webhook signature check plus a server-side re-fetch of the session (`handleStripeEvent()`), and a return-page check (`verifyPayment()`) that only accepts a session carrying this order's ID. Amount and currency compared with the order before fulfilment (`applyCheckoutSession()`)	✅
Webhook signatures verified	`stripe().webhooks.constructEvent()` on the raw body in `src/app/api/webhooks/stripe/route.ts`; forged signature → `400`	✅
Duplicate events and submissions safe	`StripeEvent` table plus conditional state transitions in `src/lib/orders.ts`; re-sent event → no second fulfilment. Retried checkout reuses the order and session (`checkoutKey`)	✅
Success updates the correct order	Order looked up from `session.metadata.orderId` and must match the stored `paymentRef`	✅
Failed / cancelled / pending handled	`markPaymentFailed()`, cancel → `/checkout?cancelled=1`, `markPaymentPending()` for SEPA	✅
Fulfilment only after verified payment	Supplier orders are created only in `markOrderPaid()`, inside a DB transaction, after Stripe reports `paid` and the amount matches	✅
Secrets not in frontend code	Stripe modules import `server-only`; no `NEXT_PUBLIC_` Stripe variable; bundle scan found no keys	✅
Tests and build pass	`npm run typecheck` clean, `npm test` 18/18, `npm run build` succeeds	✅
Real live payment	Not performed (not authorised)	🟡 Pending (Runbook §7)

Local sandbox success **does not by itself make the integration production-ready**. The blockers below must be cleared first.

2.2 Deployment blockers

#	Blocker	Why it blocks	Resolution	Owner
B1	Integration is **not committed**: about 36 changed or new files on `stripe-integration-hardening`	Nothing deployable or reviewable; no rollback point	Commit, open a PR, review, merge, tag (C-A12)	Developer
B2	Production DB lacks the new `Order` columns and the `StripeEvent` table	The new code queries these columns, so checkout and webhooks would fail	Back up, preview and apply the additive schema change before deploying (C-C1–C-C3)	Developer + DevOps
B3	Hosting, domain, secret storage, deploy and restart method unknown	Exact production commands can't be finalised	DevOps supplies the items in §12	DevOps / PM
B4	Client Stripe account ownership and live activation unconfirmed	Live keys can't be issued or payments accepted	Client completes the Launch Checklist part A	Client
B5	Legal pages under `/legal/*` are templates (README)	Stripe's review and EU consumer law expect real business details and policies	Client supplies details; legal review	Client
B6	Unknown whether production contains seeded demo accounts (published password `Demo1234!`)	Admin takeover risk	DevOps/Developer checks; remove the accounts or reset their passwords	Developer

2.3 Risks to accept or mitigate (not blockers)

#	Risk	Mitigation
R1	Disputes are only logged: order stays Paid, no alert	Client enables Stripe dispute email notifications; daily Dashboard check (Runbook §8); a follow-up development task
R2	Nothing stops sandbox keys from running in production	Mode check at every deploy (C-D5); launch checklist item
R3	No order-confirmation emails	Enable Stripe email receipts; confirm with a legal adviser whether additional confirmation is required
R4	No monitoring or alerting	Daily manual checks for 2 weeks; uptime monitor recommended
R5	No application "payments off" switch	Deploy and activate in a low-traffic window; the real-payment test follows immediately
R6	`next dev` exposes order data in debug payloads	Production must run `next build` + `next start` (or the host's equivalent), never `next dev`
R7	Partial refunds only in the Stripe Dashboard	Document in the refund procedure

3. Prerequisites

3.1 Stripe account (client): all ❓ until the client confirms

Requirement	Verified by
Client owns or controls the Stripe account	Client; developer sees the account name with their team access
Business and identity verification complete; no outstanding requirements	Dashboard home shows no activation or verification prompts
Live payments and payouts enabled	Dashboard: no "payments/payouts paused" banners
Payout bank account added and verified	Settings → Money management
Fees, payout currency and schedule reviewed	Client confirms; https://stripe.com/pricing, https://docs.stripe.com/payouts
Payment methods for live mode chosen and enabled	Settings → Payment methods (live)
Launch and real-payment test authorised	Signed Launch Checklist

The country, business type, verification status and bank details are **not assumed** anywhere in these documents.

3.2 Server and environment

Requirement	Status	How to confirm
Production hosting and deploy access	❓	DevOps (§12)
HTTPS with a valid certificate	❓	C-D1
Final production URL without redirects (www vs apex)	❓	C-D1, C-D4
Node.js ≥ 20 on the build and runtime platform	❓	C-A3 on the build host
Secure environment variable storage	❓	DevOps; §5
Outbound HTTPS to `api.stripe.com`	❓	C-D8
Inbound HTTPS from Stripe to `/api/webhooks/stripe`	❓ (app side ✅: middleware excludes `api/webhooks` from auth)	C-D4, then a live delivery
PostgreSQL production DB reachable; backup and restore available	❓	C-C1
Logs accessible to the developer	❓	C-D3
Rollback capability (previous release kept)	❓	§10
Staging environment (separate DB, sandbox keys)	❓ Recommended	PM decision

4. Production configuration

Variable	Production value	Read at	Secret	Status
`STRIPE_SECRET_KEY`	`<LIVE_KEY>`: restricted `rk_live_…` (recommended) or `sk_live_…`	Runtime (server). Client is cached per process, so **restart or redeploy after changing it**	Yes	❌ To configure
`STRIPE_WEBHOOK_SECRET`	`<LIVE_WEBHOOK_SIGNING_SECRET>` (`whsec_…` of the **live** endpoint)	Runtime (server)	Yes	❌ To configure
`NEXT_PUBLIC_APP_URL`	`https://<PRODUCTION_DOMAIN>`	**Build time** (embedded). Rebuild after changing it	No	❓
`ENABLE_DEMO_PAYMENTS`	unset or `false` (forced off in production anyway)	Runtime	No	🟡
`DATABASE_URL`, `AUTH_SECRET`	existing production values (unchanged)	Runtime; `DATABASE_URL` possibly at build (🟡; DB pages are `force-dynamic`)	Yes	❓
`MOLLIE_API_KEY`	existing value or empty (unchanged)	Runtime	Yes	❓
Publishable key (`pk_live_…`)	**Not used by this integration; do not add**	none	none	➖

4.1 Restricted key permissions

Minimum for the current code: **Checkout Sessions: Write**, **Refunds: Write**, **PaymentIntents: Read**; everything else None. 🟡 Confirm in staging with a sandbox restricted key before creating the live one. A missing permission shows up as a Stripe permission error in the logs during checkout, refund or webhook processing.

5. Configuring live credentials securely

Who obtains them: the client, or a developer with team access granted by the client, in the client's Stripe account, **live mode** → Developers → **API keys** (https://dashboard.stripe.com/apikeys). A live key you create yourself is shown **only once**; Stripe can't show it again (https://docs.stripe.com/keys).

Where they go: directly into the production secret store, by the person who copies them:

- **[Vercel]**: Project → Settings → Environment Variables, **Production** scope only, marked **Sensitive**; or the CLI (C-B2), which prompts for the value.
- **[Node host]**: the service's environment file or secret manager, readable only by the service user (C-B3).

Never: in Git, `.env.example`, frontend code, `NEXT_PUBLIC_*` variables, tickets, chat, email, deployment logs, or as a command-line argument (shell history).

Verifying without printing secrets: `/admin/payments` shows the mode and whether the webhook secret is set (C-D5). On a Node host, a one-liner prints only `live`, `TEST` or `missing` (C-D5b).

6. Production webhooks

Item	Value	Source
Route	`POST /api/webhooks/stripe`	`src/app/api/webhooks/stripe/route.ts`
Production URL	`https://<PRODUCTION_DOMAIN>/api/webhooks/stripe`, the final URL with no redirect (❓ domain)	Next.js routing
Events (exactly six)	`checkout.session.completed`, `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed`, `checkout.session.expired`, `charge.refunded`, `charge.dispute.created`	`STRIPE_WEBHOOK_EVENTS` in `src/lib/payments/stripe-events.ts`
Payload	**Snapshot** (the handler uses `constructEvent`; thin events not supported)	Code
API version	`2026-09-30.endive`	`stripe@23.0.1`

Register (live mode): Workbench → Webhooks → **Create an event destination** → Your account → Snapshot, API version `2026-09-30.endive` → select the six events → Webhook endpoint → URL above → create → reveal the signing secret → store it as `STRIPE_WEBHOOK_SECRET` (§5). Name it e.g. "olozzo production (live)".

Behaviour (✅ verified):

Situation	Response	Effect
Valid, new event	`200`	Order updated; event stored in `StripeEvent`
Duplicate event	`200`	Ignored
Missing or invalid signature, or older than 5 minutes	`400`	Ignored; logged `[stripe] invalid webhook signature`
Stripe variables not set	`501`	Ignored
DB or Stripe API error	`500`	Logged `[stripe] failed to process …`; Stripe retries for up to 3 days

Verify delivery: Stripe Dashboard (live) → Webhooks → endpoint → **Event deliveries** (status code per attempt, retry schedule, **Resend**). In the app: `/admin/payments` (latest 15 events) and each order's Payment panel.

Confirm orders are updated: after the first live payment, the event in Event deliveries shows `200`; the order shows **Paid / Processing** with the event listed in its Payment panel; supplier orders exist.

---

7. Production Server Commands — Step-by-Step Stripe Deployment

Placeholders: `<RELEASE_TAG>`, `<PREVIOUS_RELEASE_TAG>`, `<PRODUCTION_DOMAIN>`, `<APP_ROOT>`, `<ENV_FILE_PATH>`, `<SERVICE_USER>`, `<RESTART_COMMAND>`, `<SERVICE_STATUS_COMMAND>`, `<LOG_COMMAND>`, `<DEPLOYMENT_URL>`, `<BACKUP_FILE>`. Commands tagged **[Vercel]** apply only if Vercel is confirmed; **[Node host]** applies to a self-managed Node server; **[Neon]** to a Neon database. Commands are for **bash** (Git Bash on Windows works for local steps).

A. Pre-deployment inspection (developer's machine, repository root)

C-A1 · Branch and working tree 🟢 `git branch --show-current && git status --short` Permissions: repository read. Expected: the release branch; **empty status** before release (currently about 36 entries, blocker B1). Risk: none.

C-A2 · Review the release commit 🟢 `git log --oneline -10` · `git show --stat <RELEASE_TAG>` Expected: the reviewed release commit, touching Stripe, orders, checkout, admin, docs. Risk: none.

C-A3 · Runtime versions 🟢 `node -v && npm -v` Expected: Node **v20+**. Run it on the build host too, or check the host's Node setting. Risk: none.

C-A4 · Available scripts 🟢 `npm run` Expected: `build`, `start`, `typecheck`, `test`, `db:push` (and `setup` / `db:seed` / `db:reset`, which wipe the database and must never run on production). Risk: none.

C-A5 · Clean install 🟢 (local only) `npm ci` Expected: exit 0; installs exactly the lockfile versions (`stripe@23.0.1`). Risk: replaces local `node_modules`.

C-A6 · Type check 🟢 `npm run typecheck`. Expected: no output, exit 0.

C-A7 · Unit tests 🟢 `npm test`. Expected: `# fail 0` (18 passing at the time of writing).

C-A8 · Production build 🟢 `npm run build`. Expected: "Compiled successfully" and the route table. Needs a local `.env` (sandbox values are fine).

C-A9 · No secrets in browser bundles 🟢 `grep -rlE "sk_live_|rk_live_|sk_test_|rk_test_|whsec_" .next/static` Expected: **no output** (prints file names only, never values). If anything matches: stop and investigate.

C-A10 · No env files in Git 🟢 `git ls-files | grep -E "(^|/)\.env($|\.local)"` · `git check-ignore -v .env` Expected: first prints nothing; second shows `.gitignore` ignoring `.env`.

C-A11 · Deployment configuration 🟢 `git ls-files | grep -iE "dockerfile|compose|vercel\.json|ecosystem|procfile|\.service$|nginx|\.github/workflows"` Expected today: **no output** (no deploy config in the repository). If files appear later, update this guide to match them.

C-A12 · Create the release 🟡 (developer, push rights) Commit the branch, open a pull request, have it reviewed and merged, then: `git tag -a <RELEASE_TAG> -m "Stripe live integration" && git push origin <RELEASE_TAG>` Expected: the tag on the merged commit. Rollback: `git tag -d <RELEASE_TAG> && git push origin :refs/tags/<RELEASE_TAG>` (only before deployment).

B. Production environment configuration

Shell hygiene: never pass a secret as an argument. To use a secret in a local shell, read it without echo: `read -rs VAR_NAME && export VAR_NAME`, and `unset VAR_NAME` afterwards.

C-B1 · [Vercel] Link and list variables 🟢 (local; Vercel project member) `vercel link` (once) · `vercel env ls production` Expected: variable **names** for Production (values are never shown). Confirm that live Stripe variables exist **only** in Production, not Preview or Development.

C-B2 · [Vercel] Set Stripe variables 🟡 (local; Vercel project admin; client approval G3)

```
vercel env add STRIPE_SECRET_KEY production
vercel env add STRIPE_WEBHOOK_SECRET production
vercel env add NEXT_PUBLIC_APP_URL production
```

Each command **prompts** for the value. For an existing variable, run `vercel env rm <NAME> production` first, or edit it in the dashboard. Effective from the **next deployment**. Rollback: restore the previous values (keep them in the team password manager) and redeploy.

C-B3 · [Node host] Set Stripe variables 🟡 (production server; sudo)

```
sudoedit <ENV_FILE_PATH>
sudo chown <SERVICE_USER> <ENV_FILE_PATH> && sudo chmod 600 <ENV_FILE_PATH>
```

Edit the three variables in the editor. Expected: file mode `-rw-------` (`ls -l <ENV_FILE_PATH>`). How the service loads the file (systemd `EnvironmentFile=`, PM2, container secret, `.env` in `<APP_ROOT>`) is ❓ **pending DevOps confirmation**. Effective after restart (C-C5). Rollback: restore the previous file version and restart.

C. Build and deployment (approval G4, maintenance window)

C-C1 · Database backup 🟡 (DevOps; DB admin)

- **[Neon]** Console → project → create a branch or restore point named `pre-stripe-<date>`.
- **[Other Postgres]** `pg_dump "$DATABASE_URL" -Fc -f <BACKUP_FILE>.dump` (load `DATABASE_URL` with `read -rs` first).

Expected: a restorable backup. Risk: the dump contains personal data; store it encrypted and delete it after the retention period.

C-C2 · Preview the schema change 🟢 (developer; DB read access) `npx prisma migrate diff --from-url "$DATABASE_URL" --to-schema-datamodel prisma/schema.prisma --script` Expected: only `ALTER TABLE "Order" ADD COLUMN …` (7 columns), `CREATE INDEX` / `CREATE UNIQUE INDEX`, `CREATE TABLE "StripeEvent"`. **Stop if any `DROP` or type change appears**, and compare against the currently deployed schema.

C-C3 · Apply the schema change 🔴 (developer + DevOps; DB write; G4) `npx prisma db push` Expected: "Your database is now in sync with your Prisma schema". If Prisma warns about data loss: answer **No** and stop. Never use `--accept-data-loss` or `--force-reset`. Rollback: none needed for additive columns; in an emergency, restore C-C1. Run **before** C-C4/C-C5; the currently deployed code should tolerate the extra columns (🟡 verify in staging).

C-C4 · [Vercel] Deploy 🔴 (developer; Vercel deploy rights; G4) `git checkout <RELEASE_TAG> && vercel deploy --prod`. Alternatively merge to the production branch if the project deploys from Git (❓ which). Expected: deployment "Ready"; the build runs `npm run build` (including `prisma generate`). Rollback: §10.

C-C5 · [Node host] Deploy 🔴 (DevOps; app user + service control; G4)

```
cd <APP_ROOT>
git fetch --tags && git checkout <RELEASE_TAG>
npm ci
npm run build
<RESTART_COMMAND>
```

Expected: build succeeds; service restarts. If the build fails, **do not restart**; the old build keeps serving only if the build output directory isn't overwritten (❓ depends on setup). Prefer building in a fresh release directory and switching a symlink. Rollback: §10.

D. Verify the running application

C-D1 · HTTPS and status 🟢 (any machine) `curl -sSI https://<PRODUCTION_DOMAIN>/ | head -1` `echo | openssl s_client -connect <PRODUCTION_DOMAIN>:443 -servername <PRODUCTION_DOMAIN> 2>/dev/null | openssl x509 -noout -dates -subject` Expected: `HTTP/… 200`; the certificate is valid for the domain and not expiring soon.

C-D2 · Service running 🟢 **[Vercel]** `vercel inspect <DEPLOYMENT_URL>` (status Ready). **[Node host]** `<SERVICE_STATUS_COMMAND>` (❓ e.g. the systemd or PM2 status).

C-D3 · Recent logs 🟢 **[Vercel]** `vercel logs <DEPLOYMENT_URL>`. **[Node host]** `<LOG_COMMAND>`. Look for: `[checkout] payment start failed`, `[stripe] invalid webhook signature`, `[stripe] failed to process`, `StripeAuthenticationError`, `StripePermissionError`, Prisma errors. Expected: none.

C-D4 · Webhook route reachable, no redirect 🟢 `curl -sS -o /dev/null -w "%{http_code} %{redirect_url}\n" -X POST https://<PRODUCTION_DOMAIN>/api/webhooks/stripe` Expected: `400` (missing signature, no redirect). `501` means the Stripe variables are missing. `3xx` means the URL redirects, so register the final URL instead. `404` means the wrong deployment or path.

C-D5 · Payment mode (no secrets shown) 🟢 Admin login → `/admin/payments`. Expected: ✅ "Stripe is connected (**live mode**)", ✅ "Webhook signing secret is set", ✅ "Demo payments are off", ✅ public URL https. **C-D5b [Node host]**, in the same environment as the service: `node -e 'const k=process.env.STRIPE_SECRET_KEY||"";console.log(/^(sk|rk)_live_/.test(k)?"live":/^(sk|rk)_test_/.test(k)?"TEST":"missing")'` Expected: `live`.

C-D6 · Checkout reaches Stripe live 🟢 Start a checkout and stop on Stripe's page; **don't pay**. Expected: the correct total in EUR, no "Sandbox" badge. The order expires after 60 minutes, or cancel it from the admin order page.

C-D7 · No secrets in logs 🟢 Search the last 24 h of logs (C-D3) for `sk_live`, `rk_live`, `whsec_`. Expected: no matches.

C-D8 · Outbound connectivity to Stripe 🟢 (production server, [Node host]) `curl -sS -o /dev/null -w "%{http_code}\n" https://api.stripe.com/v1/charges` Expected: `401` (reachable; no key sent). A timeout means egress is blocked.

C-D9 · Payment status updates 🟢 (after the authorised real payment, Runbook §7) Stripe live → Webhooks → Event deliveries: `checkout.session.completed` `200`; the order is Paid / Processing with supplier orders; `charge.refunded` `200` after the refund.

E. Database and order consistency

Topic	Detail
Required change	Additive only: `Order` gains `checkoutKey`, `accessTokenHash`, `stockReserved` (default `false`), `paymentExpiresAt`, `paymentIntentId` (unique, nullable), `paymentFailureReason`, `refundedAt`, plus indexes on `checkoutKey` and `(paymentStatus, createdAt)`; new table `StripeEvent`
Data migration	**None required.** Existing orders get null or default values
Existing Stripe orders	Refunds still work: `refundStripe()` falls back from `paymentIntentId` to the stored Checkout Session

Topic	Detail
Existing guest order links	Orders created before this release have no access token, so guests can view them **only when signed in**; old `?session_id=` links show "Open your order from its link" (behaviour change; inform support)
Test vs live data	Live keys can't read sandbox sessions. If production ever held sandbox orders (`paymentRef` starting `cs_test_`), leave them as they are and exclude them from revenue
Forbidden on production	`npm run setup`, `db:seed`, `db:reset`, `prisma db push --force-reset` / `--accept-data-loss` (`db:seed-catalog` is additive, but isn't part of this deployment)

F. Rollback

See §10.

---

8. Production verification checklist

Before activation (live keys go live with the deploy; there's no separate switch):

☐	Check	Reference
☐	Client account active; payouts configured; G1 and G3 signed	Launch Checklist
☐	Blockers B1–B6 closed	§2.2
☐	Restricted key permissions confirmed in staging	§4.1
☐	Live key and live webhook secret stored; Production scope only	C-B1–C-B3
☐	Live webhook endpoint: six events, snapshot, `2026-09-30.endive`, final HTTPS URL	§6
☐	HTTPS valid; `NEXT_PUBLIC_APP_URL` set before the build	C-D1
☐	DB backed up; schema change applied	C-C1–C-C3
☐	Duplicate protection and refund workflow verified in sandbox	Integration Guide §12 (T6, T7, T10)
☐	Rollback target identified (`<PREVIOUS_RELEASE_TAG>` / previous deployment); monitoring owner named	§10, Runbook §8

After deployment: C-D1 to C-D8 pass, then the client-authorised real payment and refund (Runbook §7) with C-D9. Stripe Dashboard views to use: **Payments**, **Workbench → Webhooks → Event deliveries**, **Workbench → Logs**, **Balances**.

9. Money flow and settlement (summary for management)

```
Customer pays on Olozzo.com → Stripe processes the payment
→ Stripe balance shows the payment minus fees, refunds and disputes
→ After settlement, funds become available
→ Available funds are paid out on the client's payout schedule
→ Funds arrive in the client's verified bank account
```

A successful payment ("Paid" in Olozzo and in Stripe **Payments**) **doesn't mean the money is in the bank yet**. The client checks **Balances** (pending vs available) and **Payouts** (pending → in transit → paid, or failed). First payouts typically take longer (Stripe states 7–14 days after the first live payment), and timing varies by country, payment

method and risk. No fixed timeline or fee is promised here. Sources: https://docs.stripe.com/payouts, https://stripe.com/pricing. All revenue lands in the single platform account; suppliers are paid outside Stripe.

10. Rollback

Step	Action	Owner
1	**Decide:** failure confirmed by C-D checks or by customer reports; the PM informs the client	Developer + PM
2	**Record open payments:** in `/admin/orders`, note orders Unpaid / Pending created in the last 60 minutes, and live Stripe Payments of the last hour	Developer
3	**Roll back code.** [Vercel] `vercel rollback <PREVIOUS_DEPLOYMENT_URL>` or Instant Rollback in the dashboard. [Node host] `cd <APP_ROOT> && git checkout <PREVIOUS_RELEASE_TAG> && npm ci && npm run build && <RESTART_COMMAND>`	Developer / DevOps
4	**Keep the database** (additive changes). Restore the C-C1 backup only if data is corrupted, with explicit approval: it discards orders placed since the backup	DevOps
5	**Configuration:** keep the **live** key and secret (the previous release reads the same variables). **Never roll back to sandbox keys** to "recover" payments. If a new key is faulty, restore the previous live key from the secret store's history and redeploy	Developer
6	**Health:** C-D1, C-D2, C-D3, C-D4, C-D5	Developer
7	**Reconcile:** for each payment in step 2, compare Stripe with the order. If Stripe paid but the order is unpaid → admin **Sync payment**. If an order is paid twice (two live payments for one order) → refund the duplicate in Stripe. Never ask a customer to pay again until Sync payment confirms the order is unpaid	Developer
8	**Webhooks:** check Event deliveries for failures during the window. Stripe retries automatically; resend manually if needed	Developer
9	**Escalate** unresolved discrepancies: Developer → PM → client (Stripe account owner) → Stripe support from the Dashboard	PM

The previous committed code (`3f49ae8`) also identifies orders by `metadata.orderId` and handles `checkout.session.completed`, `async_payment_*` and `expired`. 🟡 Confirm in staging that it runs against the updated schema and processes sessions created by the new release.

11. Responsibility matrix

Task	Developer	DevOps / server admin	Client (Stripe owner)	Project manager
Inspect and validate implementation; close B1, B6	**R**			I
Supply hosting details (§12); close B3	C	**R**		A
Stripe onboarding, verification, bank account; close B4			**R**	I
Legal pages; close B5	C		**R**	A
Developer team access to Stripe			**R**	I
Live key and webhook endpoint creation	**R**		A	I

Task	Developer	DevOps / server admin	Client (Stripe owner)	Project manager
Store secrets in production	C	**R**		I
Backup and schema change	**R**	**R**		A
Deploy and verify	**R**	**R**		A
Authorise activation and real-payment test			**A**	R
Real-payment test and refund	**R**		C	I
Monitoring (first 2 weeks)	**R**	C	C (payouts, disputes)	I
Rollback decision	R	R	I	**A**
Record outcome and open risks	C	C	I	**R**

R = responsible · A = approves · C = consulted · I = informed

12. Information DevOps must supply before commands are final

#	Information	Used in
1	Hosting provider (Vercel confirmed or not); how production deploys (CLI or Git branch)	C-B, C-C4/C-C5
2	Production domain (www vs apex), DNS and HTTPS status	§6, C-D
3	Secret storage method and who has access	C-B
4	[Node host] `<APP_ROOT>`, `<SERVICE_USER>`, `<ENV_FILE_PATH>`, `<RESTART_COMMAND>`, `<SERVICE_STATUS_COMMAND>`, `<LOG_COMMAND>`, release-directory strategy	C-B3, C-C5, C-D
5	Database provider, backup and restore method	C-C1
6	Currently deployed commit or deployment (`<PREVIOUS_RELEASE_TAG>`)	§10
7	Whether a staging environment exists or will be created	§2.3, Mode Guide
8	Outbound firewall rules (egress to `api.stripe.com`) and inbound rules for Stripe webhooks	C-D8, C-D4

13. Troubleshooting and incident response

Issue	Owner	Diagnose	Logs	Safe remediation	Escalate to
Invalid live API key	Developer	Checkout fails for all; `/admin/payments` mode	`[checkout] payment start failed` , `StripeAuthenticationError`	Re-copy or rotate the key in Stripe; update the secret; redeploy	Client (key owner)

Issue	Owner	Diagnose	Logs	Safe remediation	Escalate to
Sandbox key deployed	Developer	`/admin/payments` shows test mode; Stripe page shows "Sandbox"; new orders have `cs_test_` refs	none	Set the live key and live webhook secret together; redeploy; cancel supplier orders of `cs_test_` orders; exclude them from revenue	PM, client
Missing variable	Developer / DevOps	Stripe missing at checkout; webhook probe `501`	none	Set the variable in the correct scope; redeploy or restart	DevOps
Webhook signature failures	Developer	Event deliveries `400`	`[stripe] invalid webhook signature`	Set the `whsec_` of **this** live endpoint; restart; resend failed events	none
Webhook delivery failing (timeouts, `3xx`, `404`, TLS)	DevOps	Event deliveries error detail	Web server or host logs	Fix the URL (final, no redirect), TLS, firewall; Stripe retries for 3 days	Stripe support
Paid in Stripe, order not updated	Developer	Compare the Stripe payment with the order	`[stripe] failed to process …`	Admin order → **Sync payment**; fix the cause; resend events	PM
Duplicate payment attempts	Developer	Two live payments for one customer or cart	Order audit trail	Retries reuse the session (`checkoutKey`); for a true duplicate charge, refund one in Stripe	Client (refund approval)
Live payment declines	Client (support) + Developer	Stripe Payments → payment → decline reason / Radar	none	Customer uses another method; review Radar rules	Stripe support
Account restricted	Client	Dashboard banner	none	Provide the requested information in the Dashboard	Stripe support
Payout delayed or failed	Client	Payouts → status and reason; Settings → Money management	none	Correct bank details or currency; wait for initial payout timing	Stripe support
Build or restart failure	DevOps + Developer	Build output; C-D2	Build log, service log	Don't switch traffic; fix or roll back (§10)	PM
Incorrect Stripe object IDs	Developer	Errors `No such checkout.session` / `payment_intent`	`[payments] verification failed`	IDs from the other mode (`cs_test_` vs `cs_live_`); expected for old sandbox orders; no Stripe object IDs are configured in this app	none
Configuration change not taking effect	Developer / DevOps	`/admin/payments` still shows old values	none	[Vercel] redeploy (env changes need a new deployment); [Node host] restart; `NEXT_PUBLIC_APP_URL` needs a **rebuild**	DevOps

Issue	Owner	Diagnose	Logs	Safe remediation	Escalate to
Payment discrepancies after rollback	Developer	§10 step 7 reconciliation	Stripe Payments vs orders	Sync payment; refund duplicates; document each case	PM → client → Stripe support

14. Official references

API keys https://docs.stripe.com/keys · Restricted keys https://docs.stripe.com/keys/restricted-api-keys · Webhooks https://docs.stripe.com/webhooks · Signatures https://docs.stripe.com/webhooks/signature · Webhook IPs https://docs.stripe.com/ips · Online payments https://docs.stripe.com/payments · Checkout https://docs.stripe.com/payments/checkout · Fulfilment https://docs.stripe.com/checkout/fulfillment · Go-live checklist https://docs.stripe.com/get-started/checklist/go-live · Testing https://docs.stripe.com/testing · Payouts https://docs.stripe.com/payouts · Refunds https://docs.stripe.com/refunds · Disputes https://docs.stripe.com/disputes · Security https://docs.stripe.com/security/guide · Pricing https://stripe.com/pricing